Safi360 — Cleaning Business SaaS Blueprint (Tanzania)

Oct 14, 2026 · @bryan

Executive summary

Build a WhatsApp-first, mobile-money-native operating system for Tanzanian cleaning companies, sold at TZS 45,000–180,000 per month. "Safi360" is a working name; swap it freely.

The product replaces three manual loops that every company studied (Bon Cleaners, Huncho, Clean Nest, Watuwenu, Mish, Moro Pure, Smile Again, Madame Sasha) runs today: quoting and booking in WhatsApp DMs, dispatching cleaners by phone, and matching M-Pesa/Tigo Pesa/Airtel Money SMS receipts to jobs by hand.

Design principles that drive every decision below:

- **WhatsApp is the front door, not a channel.** Customers already arrive via Instagram → wa.me link. The bot must live on the business's own WhatsApp number and feel like the business, not like an app.
- **Swahili first, English second.** Every customer- and cleaner-facing string ships in both from day one.
- **Deposit, not full prepayment.** Tanzanian customers rarely prepay a stranger in full. A 20–30% mobile-money deposit locks the slot and kills no-shows.
- **Cleaners have cheap Android phones and patchy data.** The field app is a PWA under 300 KB that works offline and queues photo uploads.
- **Owners are not tech-savvy.** Onboarding is done *for* them in under 48 hours, over WhatsApp and a 20-minute visit or call.

What the MVP must prove (first 90 days, 10 paying businesses):

Metric	Target
Bookings completed end-to-end through the bot or web link	≥ 40% of each tenant's jobs by month 2
Quote-to-booking conversion in the bot	≥ 25%
Deposits collected via mobile money	≥ 60% of bookings
Tenant churn after first paid month	≤ 15%

Metric	Target
Owner hours saved per week (self-reported)	⩾ 5

1. System architecture & tech stack

Ship a TypeScript modular monolith on one Postgres database with row-level tenant isolation; split services only when load proves it. This keeps a 1–3 person team fast and infra under roughly USD 60/month until ~100 tenants.

1.1 Recommended stack

Layer	Choice	Why it fits Tanzania / this team
Monorepo	pnpm + Turborepo	One repo for API, dashboard, widget, field PWA, shared types
Backend API	NestJS (Node 20+, TypeScript)	Modules map 1:1 to the domains below; DI makes per-tenant providers clean
ORM / DB	Prisma + PostgreSQL 16 with PostGIS	Relational data, money-safe integers, geo queries for nearest cleaner and zone pricing
Queue / jobs	Redis + BullMQ	Webhook processing, reminders, payment polling, retries
Business dashboard	Next.js (App Router) + Tailwind + shadcn/ui	Fast to build, SSR for the public booking page SEO
Booking widget	Same Next.js app, route `/b/[tenantSlug]`	One link the owner pastes into Instagram bio and WhatsApp catalog
Field staff app	PWA (Next.js route `/field`) with Workbox offline cache + IndexedDB upload queue	No Play Store friction; works on low-end Android and 3G
Object storage	Cloudflare R2 (S3 API)	Zero egress fees for before/after photos
Maps	Leaflet + OpenStreetMap tiles for display; Google Geocoding only on demand	WhatsApp location pins already give lat/lng for free; avoid per-call map costs

Layer	Choice	Why it fits Tanzania / this team
SMS fallback	Beem Africa or NextSMS (local), Africa's Talking as backup	Local sender-ID registration and TZ delivery rates
Hosting	Hetzner or DigitalOcean VPS + Docker Compose (Coolify for deploys) at MVP; managed Postgres (Neon/Supabase)	Lowest cost; move to managed containers at Phase 3
Auth	Phone-number OTP (WhatsApp OTP template, SMS fallback) + JWT; email/password optional for owners	Owners and cleaners live on phone numbers, not email
Observability	Sentry + Better Stack (logs, uptime)	Cheap, enough until Phase 3

Latency note: EU-hosted servers sit ~150–200 ms from Dar es Salaam, which is fine for this workload. AWS Cape Town (af-south-1) is closer but costs roughly 2–3× more; revisit only if users complain.

1.2 Multi-tenancy model

Use **shared database, shared schema, `tenant_id` on every business table, enforced by Postgres Row-Level Security (RLS).**

1. Every request resolves a tenant from one of: the JWT claim (dashboard, field app), the URL slug `/b/:slug` (widget), the WhatsApp `phone_number_id` in the webhook payload (bot), or the payment reference prefix (payment callbacks).
2. A NestJS interceptor opens a transaction and runs `SET LOCAL app.tenant_id = '<uuid>'`. All Prisma queries run inside that transaction.
3. Every tenant table carries this policy: `CREATE POLICY tenant_isolation ON bookings USING (tenant_id = current_setting('app.tenant_id')::uuid);`
4. The Super Admin connects with a separate DB role that has `BYPASSRLS`, used only by the `/platform` admin module.
5. Add an automated test that hits every endpoint as tenant A and asserts zero rows from tenant B.

White-labelling: each tenant gets `tenant.branding` (logo, primary colour, display name, Swahili/English greeting, terms text). The widget lives at `book.safi360.co.tz/<slug>` at MVP, custom domains (`book.boncleaners.co.tz`) via Cloudflare for SaaS custom hostnames in Phase 2. The bot is white-label by default because it runs on the tenant's own WhatsApp number.

1.3 WhatsApp integration

Use the **Meta WhatsApp Cloud API directly**, with the platform registered as a Meta **Tech Provider** so each cleaning company connects its *own* number through **Embedded Signup** (a popup inside the dashboard). Avoid Twilio/Infobip at MVP: they add a per-message markup on top of Meta's fees for no feature you need.

Setup steps:

1. Create a Meta Business Manager for the platform, verify the business, create a Meta App with the WhatsApp product, and apply for Tech Provider status.
2. Implement Embedded Signup in the dashboard. On success, store per tenant: `waba_id`, `phone_number_id`, a system-user access token (encrypted with AES-256-GCM, key in env/KMS).
3. Subscribe the app to the tenant's WABA webhooks. One webhook endpoint `POST /webhooks/whatsapp` serves all tenants; route by `entry[].changes[].value.metadata.phone_number_id`.
4. Verify the `X-Hub-Signature-256` header, respond `200` within 1 second, and push the payload onto a BullMQ queue. Never process inline.
5. Deduplicate on `messages[].id` (Meta retries).
6. Use interactive messages: reply buttons (⩽3 options), list messages (⩽10 rows, for the service menu), location-request messages (one-tap "Send location"), and **WhatsApp Flows** for the multi-field quote form.
7. Pre-approve message templates per tenant at onboarding (Swahili + English): `booking_confirmed`, `deposit_request`, `cleaner_en_route`, `job_completed_feedback`, `maintenance_reminder`, `payment_received`.

Number problem: most owners already use their number in the WhatsApp Business app. Meta's coexistence option lets a number run on the Business app and the Cloud API together, so the owner keeps chatting manually while the bot handles bookings. Confirm current eligibility for Tanzanian numbers during setup; if a number is not eligible, register a second dedicated "bookings" number.

Cost: customer-initiated service messages inside the 24-hour window are free under Meta's current per-message pricing, and the booking flow lives almost entirely inside that window. Utility templates (reminders sent later) and marketing templates are paid per message at Africa-region rates; check Meta's current rate card before setting tier limits.

1.4 Mobile money payment pipeline

Integrate **Selcom** first (widest coverage: M-Pesa, Mixx by Yas (formerly Tigo Pesa), Airtel Money, HaloPesa, bank cards, plus till/Lipa Namba) and **AzamPay** second as a fallback and for tenants who already hold an AzamPay account. Keep DPO as a Phase 3 option for card-

heavy corporate clients.

Money flow — tenant is merchant of record. Each cleaning company signs its own merchant account with Selcom or AzamPay; the platform stores their API credentials encrypted and calls the gateway on their behalf. Funds settle straight to the company. The platform never holds customer money, which avoids needing a Bank of Tanzania payment-service licence. For the first tenants who have no merchant account yet, help them apply during onboarding and fall back to manual confirmation (Section 3d) meanwhile.

Build a provider adapter so the rest of the code never knows which gateway is used:

```
interface PaymentProvider {
  initiatePush(req: { tenantId: string; amountTzs: number; msisdn: string;
reference: string; description: string }): Promise<{ providerRef: string;
status: 'PENDING' }>;
  createCheckoutLink(req: { tenantId: string; amountTzs: number; reference:
string; customerPhone: string; redirectUrl: string }): Promise<{ url: string;
providerRef: string }>;
  verifyWebhook(headers: Record<string,string>, rawBody: Buffer): boolean;
  parseWebhook(rawBody: Buffer): { reference: string; providerRef: string;
status: 'SUCCESS'|'FAILED'; amountTzs: number; channel: string; paidAt: Date
};
  queryStatus(providerRef: string): Promise<'PENDING'|'SUCCESS'|'FAILED'>;
}
// implementations: SelcomProvider, AzamPayProvider, ManualProvider
```

Pipeline:

1. Booking reaches `AWAITING_PAYMENT` → `payments` row with status `PENDING` and a unique `reference` derived from the ticket, like `UBC2026000421D` (ticket without dashes + attempt number + D for deposit / B for balance; ≤ 20 chars, alphanumeric — wallets truncate long or punctuated refs).
2. Call `initiatePush` → customer gets a USSD PIN prompt on their phone. Also send a checkout link as backup (cards, or if the push times out).
3. Gateway webhook → verify signature → idempotent update by `reference` → move booking from `AWAITING_PAYMENT` to `CONFIRMED` → send WhatsApp confirmation quoting the ticket number.
4. A BullMQ job polls `queryStatus` at 30 s, 2 min, 10 min for any payment still `PENDING` (webhooks get lost).
5. Daily job pulls the gateway transaction report and auto-matches it against `payments`; unmatched rows go to a reconciliation inbox. A payment that arrives after its booking `EXPIRED` is stored as unmatched and raised in the exceptions queue, never silently applied.

Store all amounts as integers in TZS (no decimals). Phone numbers always in E.164 (`+2557XXXXXXXX`); normalise `07…` , `7…` and `255…` input on entry.

1.5 Compliance checklist

- Register with Tanzania's Personal Data Protection Commission as a data controller/processor under the Personal Data Protection Act 2022; publish a Swahili + English privacy notice.
- Register an SMS sender ID with the SMS provider (TCRA rules apply).
- Let tenants issue EFD/VFD receipts later (Phase 3 integration); until then, invoices state they are not tax receipts.

2. Core modules & database schema

Twelve NestJS modules sit on about 20 tables; every business table carries `tenant_id` and an RLS policy.

2.1 Roles and permissions

Role	Logs in via	Can do	Cannot do
Super Admin (platform)	Email + password + TOTP, `/platform`	Create/suspend tenants, set plans, view platform metrics, impersonate a tenant (logged)	Edit a tenant's bookings without impersonation
Business Owner	Phone OTP or email	Everything in their tenant: catalog, pricing, staff, payouts setup, reports, subscription	See other tenants
Business Manager / Dispatcher	Phone OTP	Bookings, dispatch, customers, payments view, reconciliation	Billing, delete tenant, change gateway keys
Field Staff (cleaner / team lead)	Phone OTP into PWA	See own assigned jobs, update status, upload photos, collect cash with reason, mark items done	See prices of other jobs, customer list, revenue
End Customer	No account; identified by phone number (WhatsApp or OTP on web)	Book, pay deposit, reschedule/cancel within policy, rate job, view own history via magic link	—

Implement as `membership.role` enum plus a small permission map in code (`can(role, 'booking.assign')`); no generic RBAC tables at MVP.

2.2 Modules

`auth` · `tenants` (branding, settings, plan) · `catalog` (services, pricing rules, add-ons) · `quotes` (pricing engine) · `availability` (slots, capacity, zones) · `bookings` · `dispatch` · `field` (status, photos, checklists) · `customers` (CRM, tags, recurrence) · `payments` (provider adapters, reconciliation) · `messaging` (WhatsApp, SMS, templates, bot state machine) · `billing` (tenant subscriptions) · `reports` · `platform` (super admin).

2.3 Pricing model (the part most systems get wrong)

Cleaning prices in Tanzania are quoted per unit, per size or per area, often with a minimum. Model each service with one `pricing_type` and a JSON rule, so the quote engine is one pure function.

pricing_type	Example service	Rule JSON	Quote formula
PER_UNIT	Sofa cleaning	{"unit":"seat","price":15000,"min_charge":45000}	max(qty × price, min)
PER_VARIANT	Mattress / carpet by size	{"variants":[{"key":"5x6","price":35000},{"key":"6x6","price":40000}]}	Σ variant price × qty
PER_AREA	Post-construction, office	{"unit":"sqm","tiers":[{"upTo":100,"price":2500},{"upTo":null,"price":2000}],"min_charge":150000}	tiered area × rate, then min
PER_ROOM	Residential deep clean	{"rooms":{"bedroom":30000,"bathroom":20000,"kitchen":35000,"sitting":30000}}	Σ rooms
FLAT	Fumigation of 3-bedroom house	{"price":120000}	price
CUSTOM_QUOTE	Large commercial	{}	goes to owner for manual quote

Modifiers apply after the base: add-ons (stain treatment, odour removal), zone/travel surcharge by distance from the tenant's base (`{"zones":[{"maxKm":10,"fee":0},{"maxKm":25,"fee":10000}]}`), weekend/evening surcharge, and promo codes. Return a line-itemised quote so the customer sees *why* it costs what it costs — this is what replaces the DM negotiation.

2.4 Schema (PostgreSQL DDL)

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TYPE booking_status AS ENUM (

'REQUEST_RECEIVED','AWAITING_REVIEW','AWAITING_CUSTOMER_APPROVAL','AWAITING_PAYMENT',

'CONFIRMED','ASSIGNED','EN_ROUTE','ARRIVED','IN_PROGRESS','COMPLETED','CLOSED',
  'CANCELLED','QUOTE_REJECTED','NO_SHOW','EXPIRED'      -- terminal outcomes, reason required
);
CREATE TYPE booking_source AS ENUM (
```

```
'website','booking_widget','whatsapp','instagram','facebook','phone','walk_in
','referral','admin_created'
);

-- ===== Platform level (no RLS) =====
CREATE TABLE plans (
  id            text PRIMARY KEY,              -- 'starter','growth','pro'
  name          text NOT NULL,
  price_tzs     integer NOT NULL,
  limits        jsonb NOT NULL                 --
{"staff":5,"bookings_month":150,"wa_templates_month":300}
);

CREATE TABLE tenants (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug          text UNIQUE NOT NULL,
  name          text NOT NULL,
  phone         text NOT NULL,
  city          text,
  base_location geography(Point,4326),
  ticket_prefix text UNIQUE NOT NULL,          -- e.g. 'UBC'; locked once
the first booking exists
  branding      jsonb NOT NULL DEFAULT '{}',   -- logo_url, color,
greeting_sw, greeting_en
  settings      jsonb NOT NULL DEFAULT '{}',   -- deposit_pct, cancel_hours,
working_hours, languages
  plan_id       text REFERENCES plans(id),
  status        text NOT NULL DEFAULT 'TRIAL',  -- TRIAL, ACTIVE, PAST_DUE,
SUSPENDED
  trial_ends_at timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  phone      text UNIQUE NOT NULL,             -- E.164
  name       text,
  email      text UNIQUE,
  locale     text NOT NULL DEFAULT 'sw',
  is_super_admin boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE memberships (                     -- user <-> tenant with role
  tenant_id uuid REFERENCES tenants(id),
  user_id   uuid REFERENCES users(id),
```

```
  role      text NOT NULL,                        -- OWNER, MANAGER, STAFF
  active    boolean NOT NULL DEFAULT true,
  PRIMARY KEY (tenant_id, user_id)
);

CREATE TABLE tenant_integrations (
  tenant_id   uuid REFERENCES tenants(id),
  kind        text NOT NULL,                      -- WHATSAPP, SELCOM, AZAMPAY,
SMS
  config      jsonb NOT NULL,                     -- non-secret ids (waba_id,
phone_number_id)
  secret_enc  bytea,                              -- AES-256-GCM encrypted
token/keys
  status      text NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (tenant_id, kind)
);
CREATE UNIQUE INDEX ON tenant_integrations ((config->>'phone_number_id'))
WHERE kind='WHATSAPP';

-- ===== Tenant level (RLS on all) =====
CREATE TABLE service_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  name_sw text NOT NULL, name_en text NOT NULL,
  sort int NOT NULL DEFAULT 0
);

CREATE TABLE services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  category_id uuid REFERENCES service_categories(id),
  name_sw text NOT NULL, name_en text NOT NULL,
  description_sw text, description_en text,
  pricing_type text NOT NULL,                     -- PER_UNIT, PER_VARIANT,
PER_AREA, PER_ROOM, FLAT, CUSTOM_QUOTE
  pricing_rule jsonb NOT NULL,
  duration_rule jsonb NOT NULL,                   --
{"base_min":60,"per_unit_min":20}
  crew_size int NOT NULL DEFAULT 1,
  recurrence_days int,                            -- e.g. 90 for quarterly sofa
reminder
  photo_url text,
  active boolean NOT NULL DEFAULT true
);

CREATE TABLE service_addons (
```

```
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  service_id uuid REFERENCES services(id),
  name_sw text NOT NULL, name_en text NOT NULL,
  price_tzs integer NOT NULL,
  per_unit boolean NOT NULL DEFAULT false
);

CREATE TABLE service_zones (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  name text NOT NULL,                          -- 'Mbezi/Kimara', 'Kisauni'
  area geography(Polygon,4326),                -- optional; else distance
bands from base
  travel_fee_tzs integer NOT NULL DEFAULT 0,
  serviceable boolean NOT NULL DEFAULT true
);

CREATE TABLE customers (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  phone text NOT NULL,
  name text,
  locale text NOT NULL DEFAULT 'sw',
  source text,                                 -- WHATSAPP, WEB, INSTAGRAM,
MANUAL
  tags text[] NOT NULL DEFAULT '{}',
  marketing_opt_in boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, phone)
);

CREATE TABLE customer_addresses (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  customer_id uuid NOT NULL REFERENCES customers(id),
  label text,                                  -- 'Nyumbani', 'Ofisi'
  location geography(Point,4326) NOT NULL,
  landmark text,                               -- 'Karibu na Shell Mbezi,
geti jeusi'
  area_name text,                              -- reverse-geocoded or typed
  zone_id uuid REFERENCES service_zones(id)
);

CREATE TABLE staff (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  user_id uuid REFERENCES users(id),
  display_name text NOT NULL,
  skills uuid[] NOT NULL DEFAULT '{}',          -- service ids they can do
  home_zone_id uuid REFERENCES service_zones(id),
  is_team_lead boolean NOT NULL DEFAULT false,
  pay_type text,                                -- PER_JOB, DAILY, MONTHLY
  pay_rate_tzs integer,
  active boolean NOT NULL DEFAULT true
);

CREATE TABLE staff_availability (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  staff_id uuid NOT NULL REFERENCES staff(id),
  weekday smallint,                             -- 0-6, null when date-
specific
  date date,                                    -- leave/off days override
  start_time time, end_time time,
  is_off boolean NOT NULL DEFAULT false
);

CREATE TABLE quotes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  customer_id uuid REFERENCES customers(id),
  input jsonb NOT NULL,                         -- {"items":
[{"service_id":..,"qty":5,"variant":null}],"address_id":..}
  lines jsonb NOT NULL,                         -- itemised result
  subtotal_tzs integer NOT NULL,
  travel_fee_tzs integer NOT NULL DEFAULT 0,
  discount_tzs integer NOT NULL DEFAULT 0,
  total_tzs integer NOT NULL,
  est_duration_min integer NOT NULL,
  expires_at timestamptz NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE bookings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  ticket_ref text NOT NULL,                  -- e.g. UBC-2026-00042;
immutable, never reused
  customer_id uuid NOT NULL REFERENCES customers(id),
  address_id uuid NOT NULL REFERENCES customer_addresses(id),
  quote_id uuid REFERENCES quotes(id),
```

```
  booking_source booking_source NOT NULL,                          -- channel
the booking came from; RECURRING tracked via recurrence_parent_id
  status booking_status NOT NULL DEFAULT 'REQUEST_RECEIVED',  -- see Section
3 diagram
  scheduled_start timestamptz NOT NULL,
  scheduled_end timestamptz NOT NULL,
  total_tzs integer NOT NULL,
  deposit_tzs integer NOT NULL DEFAULT 0,
  paid_tzs integer NOT NULL DEFAULT 0,
  notes text,
  recurrence_parent_id uuid REFERENCES bookings(id),
  outcome_reason text,                          -- cancelled / quote rejected
/ no-show / expired
  flags text[] NOT NULL DEFAULT '{}',           -- RESCHEDULED, NEEDS_REVIEW
(status unchanged)
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, ticket_ref)
);
CREATE INDEX ON bookings (tenant_id, scheduled_start);
CREATE INDEX ON bookings (tenant_id, status);

CREATE TABLE booking_items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  booking_id uuid NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
  service_id uuid NOT NULL REFERENCES services(id),
  variant text, qty numeric NOT NULL,           -- numeric for sqm
  unit_price_tzs integer NOT NULL,
  line_total_tzs integer NOT NULL,
  addons jsonb NOT NULL DEFAULT '[]'
);

CREATE TABLE assignments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  booking_id uuid NOT NULL REFERENCES bookings(id),
  staff_id uuid NOT NULL REFERENCES staff(id),
  is_lead boolean NOT NULL DEFAULT false,
  assigned_by uuid REFERENCES users(id),
  assigned_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (booking_id, staff_id)
);

CREATE TABLE job_events (                      -- append-only status timeline
  id bigserial PRIMARY KEY,
  tenant_id uuid NOT NULL REFERENCES tenants(id),
```

```
  booking_id uuid NOT NULL REFERENCES bookings(id),
  actor_user_id uuid REFERENCES users(id),
  type text NOT NULL,                          -- ASSIGNED, EN_ROUTE,
ARRIVED, STARTED, COMPLETED, ISSUE, CASH_COLLECTED
  location geography(Point,4326),
  payload jsonb NOT NULL DEFAULT '{}',
  occurred_at timestamptz NOT NULL,            -- device time (offline-safe)
  received_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE job_photos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  booking_id uuid NOT NULL REFERENCES bookings(id),
  booking_item_id uuid REFERENCES booking_items(id),
  kind text NOT NULL,                          -- BEFORE, AFTER, ISSUE
  storage_key text NOT NULL,
  taken_at timestamptz NOT NULL,
  location geography(Point,4326),
  uploaded_by uuid REFERENCES users(id)
);

CREATE TABLE payments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  booking_id uuid REFERENCES bookings(id),
  reference text NOT NULL,                     -- UBC2026000421D (ticket +
attempt + D/B)
  purpose text NOT NULL,                       -- DEPOSIT, BALANCE, FULL
  provider text NOT NULL,                      -- SELCOM, AZAMPAY, CASH,
MANUAL_MOMO
  channel text,                                -- MPESA, MIXX, AIRTEL,
HALOPESA, CARD, CASH
  msisdn text,
  amount_tzs integer NOT NULL,
  status text NOT NULL,                        -- PENDING, SUCCESS, FAILED,
EXPIRED, REFUNDED
  provider_ref text,
  provider_txn_id text,                        -- e.g. M-Pesa receipt code
  raw jsonb,
  matched_by text,                             -- WEBHOOK, POLL, REPORT,
MANUAL
  confirmed_by uuid REFERENCES users(id),
  created_at timestamptz NOT NULL DEFAULT now(),
  paid_at timestamptz,
  UNIQUE (tenant_id, reference)
```

```
);
CREATE UNIQUE INDEX ON payments (provider, provider_txn_id) WHERE
provider_txn_id IS NOT NULL;

CREATE TABLE conversations (                    -- bot state per customer
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  customer_id uuid NOT NULL REFERENCES customers(id),
  channel text NOT NULL DEFAULT 'WHATSAPP',
  state text NOT NULL,                           -- see Section 4
  context jsonb NOT NULL DEFAULT '{}',           -- cart, chosen slot, quote
id
  handed_off boolean NOT NULL DEFAULT false,     -- human took over
  last_inbound_at timestamptz,                   -- drives 24h service window
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, customer_id, channel)
);

CREATE TABLE messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  conversation_id uuid REFERENCES conversations(id),
  direction text NOT NULL,                       -- IN, OUT
  provider_msg_id text UNIQUE,
  type text NOT NULL,                            -- text, interactive,
location, template, image
  body jsonb NOT NULL,
  status text,                                   -- sent, delivered, read,
failed
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE scheduled_notifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  customer_id uuid REFERENCES customers(id),
  booking_id uuid REFERENCES bookings(id),
  template text NOT NULL,                        -- maintenance_reminder,
day_before_reminder
  send_at timestamptz NOT NULL,
  status text NOT NULL DEFAULT 'QUEUED',         -- QUEUED, SENT, FAILED,
CANCELLED
  vars jsonb NOT NULL DEFAULT '{}'
);

CREATE TABLE reviews (
```

```
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  booking_id uuid UNIQUE NOT NULL REFERENCES bookings(id),
  rating smallint NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE subscriptions (                    -- the tenant paying the
platform
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  plan_id text NOT NULL REFERENCES plans(id),
  period_start date NOT NULL, period_end date NOT NULL,
  amount_tzs integer NOT NULL,
  status text NOT NULL,                          -- PENDING, PAID, OVERDUE
  payment_ref text
);

CREATE TABLE audit_log (
  id bigserial PRIMARY KEY,
  tenant_id uuid, actor_user_id uuid,
  action text NOT NULL, entity text, entity_id uuid,
  diff jsonb, created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE tenant_counters (                 -- one row per tenant per year;
locked FOR UPDATE
  tenant_id uuid REFERENCES tenants(id),
  year int NOT NULL,
  last_value int NOT NULL DEFAULT 0,
  PRIMARY KEY (tenant_id, year)
);

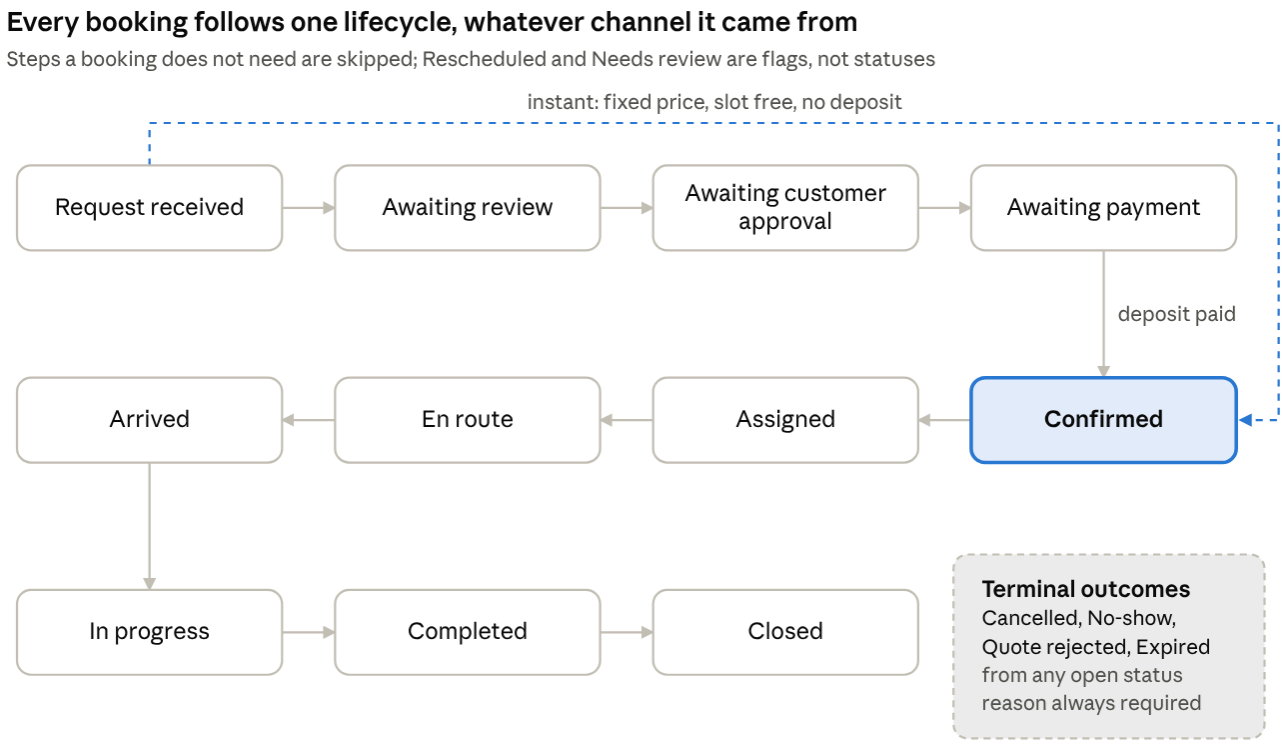
-- RLS: run for every table that has tenant_id (except audit_log, platform-
written)
-- ALTER TABLE <t> ENABLE ROW LEVEL SECURITY;
-- CREATE POLICY tenant_isolation ON <t>
--   USING (tenant_id = current_setting('app.tenant_id')::uuid)
--   WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

Key relationships: a tenant has many services, staff, customers and bookings. A customer (one per verified phone per tenant) has many addresses, many bookings and one bot conversation per channel. A booking is one ticket: it has many booking items (one per service line), many assignments (crew), an append-only job-event timeline, many photos and many payments (deposit + balance).

Ticket numbers: `ticket_ref` = `{tenant.ticket_prefix}-{YYYY}-{5-digit counter}`, e.g. `UBC-2026-00042`. It is generated inside the booking-insert transaction by locking that tenant's `tenant_counters` row for the year (`SELECT … FOR UPDATE`, increment, format). Cancelled and deleted bookings keep their numbers (soft delete only), so a number is never reused. Prefixes are unique platform-wide, so a ticket on its own identifies the tenant. The public tracking lookup still requires the phone code before showing anything (see Section 3a). A ticket identifies a booking, not a customer: repeat customers get a new ticket each time.

3. Functional workflows

Every booking moves through one state machine; the four flows below are the actors pushing it forward.



booking lifecycle · 11 statuses, 4 outcomes

Only `BookingStateMachine.transition()` may move a booking between statuses, and every transition writes a `job_events` row recording who made it, when, and why. The same lifecycle and labels are used in the Stitch UI prompt, the operations portal, the customer tracking page and the cleaner app.

3a. Customer booking flow

All channels call one `create_booking` service. The website, booking widget, WhatsApp bot and staff "Create booking" form differ only in how they collect input and which `booking_source` they record.

1. Customer arrives through any channel: the website or widget at `book.safi360.co.tz/<slug>` (a `?src=instagram|facebook|qr` parameter sets the source), the wa.me link (bot), or by phone or walk-in (staff use "Create booking").
2. The system finds or creates the customer by verified phone (one record per phone per tenant). Unverified or conflicting matches go to the duplicate-review list instead of being merged.
3. Customer or staff pick service(s), quantities or measurements, optional photos → the `quotes` engine returns an itemised price in under 1 s, or marks the job "inspection / quote required".
4. Location: map pin plus region, district, ward, street and a required landmark → zone and travel fee resolved. Outside all zones, the bot or page offers a callback.
5. `availability` returns open slots that fit `est_duration_min` and crew size. The chosen slot is held for 15 minutes (Redis key with TTL).
6. Submit → server recalculates price and availability, checks the idempotency key (a repeat submission returns the existing ticket), allocates `ticket_ref` (e.g. `UBC-2026-00042`) and creates the booking at `REQUEST_RECEIVED`.
7. Routing by rules:
   - Standard fixed-price service, slot available, no deposit required → `CONFIRMED` immediately.
   - Deposit required → `AWAITING_PAYMENT` → push payment → `CONFIRMED` on webhook.
   - Needs a human check (custom quote, inspection, unusual scope) → `AWAITING_REVIEW` → staff send a quote → `AWAITING_CUSTOMER_APPROVAL` → approved → `AWAITING_PAYMENT` or `CONFIRMED`.
8. Confirmation message on WhatsApp/SMS quotes the ticket number and says plainly whether this is a confirmed appointment or a request under review. It includes a signed tracking link.
9. Day-before reminder at 18:00 and a 2-hours-before reminder are queued in `scheduled_notifications`. Every notification's delivery status (queued, sent, delivered, failed) is stored from provider callbacks.

Customer tracking (Track My Booking): the customer enters the ticket number and registered phone → the API always replies "code sent" (it never reveals whether the pair matched) → 6-digit code → a short-lived token scoped to that booking → a customer-safe view: status timeline, services, appointment, total, paid and balance, address, cleaner first

name, and the actions allowed by policy (approve quote, reschedule, cancel, pay balance, receipt, review). Lost tickets: phone + code → list of that phone's bookings with this company only. Signed links in notifications skip the code step.

3b. Admin dispatch flow

1. The "Needs action" panel and ticket list show new requests (`REQUEST_RECEIVED` , `AWAITING_REVIEW`), unassigned confirmed bookings, and exceptions. Each ticket shows its source chip.
2. Staff review requests: adjust scope or price and send a quote (`AWAITING_CUSTOMER_APPROVAL`), confirm, or cancel with a reason.
3. The calendar shows confirmed bookings in a time × cleaner grid, with an unassigned tray, travel buffers, working hours and leave.
4. Assignment is manual at MVP. The drawer lists cleaners filtered by skill and availability for the whole window, sorted by distance from their previous job and by today's load. Conflicts block the save with an explanation ("Juma is on UBC-2026-00041 until 11:30").
5. Assigning moves the booking to `ASSIGNED` and notifies the cleaner. The customer gets the cleaner's first name the day before. Reassigning notifies both cleaners, and a reschedule records a reason and sets the `RESCHEDULED` flag.
6. If a cleaner becomes unavailable after assignment (leave added, sick), affected bookings get the `NEEDS_REVIEW` flag and appear in the exceptions queue.
7. Phone, WhatsApp and walk-in bookings go through the same "Create booking" form. Staff search for the customer by phone or name, set the booking source, and save as Draft, Request or Confirmed. The ticket appears immediately with Copy and Send-to-customer actions.

3c. Field staff execution flow

1. Cleaner opens the PWA (OTP once, then a 30-day session) and sees today's jobs in order, each with map link, landmark, customer first name, items, and balance to collect.
2. **Start trip** → `EN_ROUTE` + GPS point → customer gets "Mtaalamu wako yuko njiani" (your cleaner is on the way) with ETA.
3. **Arrived** → `ARRIVED` (GPS must be within 300 m of the address, else a reason is required).
4. **Before photos**: at least 1 per booking item, camera-only (no gallery) to prevent reuse; compressed to ⩽ 300 KB in the browser before queueing.
5. **Start job** → `IN_PROGRESS` ; optional checklist per service.
6. **After photos**: same count as before photos; app blocks completion until they exist.

7. **Collect balance**: tap "Send payment request" (push to customer phone) or "Cash received" (records `CASH` payment, flagged for owner review).
8. **Complete** → `COMPLETED` → customer gets the before/after collage, receipt and a 1–5 rating request.
9. Offline: every action writes to IndexedDB with device timestamp and syncs when online; `job_events.occurred_at` keeps true order.

3d. Payment reconciliation

Every shilling must land in exactly one `payments` row tied to one booking, with a clear source of truth.

1. **Gateway payments (automatic):** webhook → match by `reference` → status `SUCCESS`, `matched_by='WEBHOOK'`. Poll as backup. Booking `paid_tzs` recomputed as the sum of successful payments (never incremented in place).
2. **Direct-to-till payments (semi-automatic):** many customers will still pay the owner's Lipa Namba or personal number. The owner (or dispatcher) forwards or pastes the M-Pesa/Mixx/Airtel confirmation SMS into the dashboard or to the platform's WhatsApp number. A parser extracts transaction ID, amount, payer name/number and time, then proposes matches: same phone number and amount within ±48 h of an open balance. One tap confirms. `provider_txn_id` unique index blocks double-counting.
3. **Cash:** recorded by the cleaner, appears in a "Cash to hand over" list per cleaner per day; owner marks it received.
4. **Daily close (automatic, 23:00):** pull gateway transaction reports, flag (a) gateway transactions with no booking, (b) completed bookings with an unpaid balance, (c) cash not yet handed over. Owner gets a WhatsApp summary: "Leo: kazi 7, TZS 685,000 imelipwa, TZS 90,000 bado" (Today: 7 jobs, TZS 685,000 paid, TZS 90,000 outstanding).
5. **Refunds:** recorded as a negative-purpose payment with a reason; actual refund done by the owner in their wallet/gateway at MVP.

Reports built on this: revenue by day/service/channel/cleaner, outstanding balances, deposit conversion rate, average job value, repeat-customer rate.

4. Low-friction customer experience

Target: a first-time customer goes from tapping the Instagram link to a paid, confirmed booking in **under 90 seconds and 7 taps, with zero typing except a landmark**.

4.1 WhatsApp bot conversation (Swahili default, English on request)

Bot states (`conversations.state`): `GREETING → MENU → ITEMS → LOCATION → SLOT → REVIEW → AWAITING_PAYMENT → CONFIRMED`, plus `HUMAN` at any point. Every message carries a "Ongea na mtu / Talk to a person" option.

1. **Greeting** (any first message, including "Habari" or an Instagram reply). Branded text + list message:

   > Karibu **Bon Cleaners**! 👋 Tunasafisha sofa, mazulia, magodoro na nyumba nzima. Chagua huduma: [Angalia huduma] List rows come from `services` (≤10 per section; sections = categories). Last row: "English" toggles locale.

2. **Items.** Use a **WhatsApp Flow** (native form screen) with steppers per item: "Sofa — viti vingapi? [− 5 +]", "Godoro — ukubwa [5×6 | 6×6]", "Mazulia — idadi". For per-area services: ranges as buttons ("Chini ya 50 m²", "50–100 m²", "100–200 m²", "Zaidi") — customers do not know exact square metres. If Flows are unavailable on the device, fall back to reply buttons with quick counts (1–2 / 3–5 / 6+), then one numeric text reply.
3. **Instant quote** shown before asking for anything else, so price never blocks the conversation:

   > Sofa viti 5 × 15,000 = **75,000** Kuondoa madoa (add-on) = 10,000 Jumla kabla ya usafiri: **TZS 85,000** [Endelea] [Badilisha] [Ongea na mtu]

4. **Location.** Send a location-request message (one big "Tuma Location" button). Then one text prompt: "Alama ya karibu? (mf. karibu na Shell Mbezi, geti jeusi)". Returning customers see saved addresses as buttons instead: [Nyumbani – Mbezi] [Mahali pengine].
5. **Slot.** List message with the next 6 available slots, grouped by day: "Kesho Jumamosi 9:00 asubuhi", "Kesho 2:00 mchana"… Use Swahili time-of-day words (asubuhi, mchana, jioni) alongside the clock.
6. **Review + pay.** One summary card with total including travel fee, deposit and balance:

   > Jumla: TZS 90,000 · Amana sasa: **TZS 27,000** · Salio siku ya kazi: TZS 63,000 Tumia namba hii kulipa? **0754 XXX 123** [Ndiyo, lipa] [Namba nyingine] Tapping "Ndiyo, lipa" fires the USSD push. Bot says: "Angalia simu yako, weka PIN ya M-Pesa ✅". Also attaches a checkout link for card or another wallet.

7. **Confirmation** on webhook (or on the 2-minute poll): ticket number (e.g. UBC-2026-00042), date, time window, "Utapokea jina la mtaalamu siku moja kabla". If the push fails or times out: [Jaribu tena] [Lipa kwa Lipa Namba] [Ongea na mtu].

8. **After the job:** before/after collage + "Tupe nyota 1–5" reply buttons (3 buttons: ⭐⭐⭐⭐⭐ / ⭐⭐⭐ / ⭐ then a follow-up for detail). 4–5 stars → ask for a Google review link or an Instagram story tag; 1–3 → alert the owner instantly.
9. **Re-engagement:** at `recurrence_days` after completion (e.g. 90 days for sofas, 30 for fumigation), utility template: "Imepita miezi 3 tangu sofa zako zisafishwe. Tupange tena? [Ndiyo, nipangie] [Baadaye]". "Ndiyo" re-opens the flow with the last order and address prefilled — one tap to quote.

Bot rules that prevent frustration:

- Free-text understanding at MVP is keyword-based (Swahili + English synonyms: "kochi", "sofa", "makochi" → sofa service). Phase 2 adds an LLM intent/entity extractor for messages like "nataka kusafisha sofa 3 na zulia moja kesho" that fills the cart directly.
- Any unrecognised message twice in a row, or the words "mtu", "agent", "bei gani kweli" → hand off: set `handed_off=true`, notify the owner, stop the bot for that chat for 12 h. The owner replies from the dashboard inbox or (with coexistence) from their normal WhatsApp Business app.
- Session timeout: if a customer goes quiet for 30 min mid-flow, send one nudge; after 23 h, the cart is kept but the next message restarts politely ("Tuendelee na sofa 5?").
- Never send more than 2 messages in a row without a customer action.

4.2 Web booking widget (`/b/<slug>`)

For customers who click from the Instagram bio, Google Maps listing or a shared link. One page, mobile-first, no login.

1. **Header:** tenant logo, rating, "Bei wazi, bila kubishana" (clear prices, no haggling). Language toggle.
2. **Service cards** with photo and "kuanzia TZS 15,000" (from…). Tap to add; steppers inline. A sticky bottom bar shows the running total, updating live as quantities change.
3. **Location:** "Tumia location yangu" button (browser geolocation) → draggable pin on a Leaflet map, landmark field. Fallback: search box with Dar/Zanzibar/Morogoro neighbourhood autocomplete from a seeded list (Mbezi, Kimara, Sinza, Kisauni, Mazizini…).
4. **Date strip** (next 14 days as chips) → available time chips for the chosen day.
5. **Phone number** (single field, prefilled +255) → OTP via WhatsApp, SMS fallback. Name optional.
6. **Pay deposit:** wallet selector icons (M-Pesa, Mixx by Yas, Airtel Money, HaloPesa, Card) → USSD push to that number with a live "waiting for PIN" screen that polls status every 3 s for 90 s.

7. **Success page** + the same WhatsApp confirmation, so the conversation continues in WhatsApp from here on.

Performance budget: first load ⩽ 150 KB JS, Lighthouse mobile ⩾ 90, works on 3G. Preload the tenant catalog as JSON in the page (SSR) so no extra round-trip before the menu shows.

5. Monetization & business model

Charge a flat monthly subscription in three tiers from TZS 45,000 to TZS 180,000, sold on one promise: **"one extra sofa job a week pays for the whole system."** A typical sofa job is TZS 60,000–100,000, so even Starter pays back with a single recovered booking.

5.1 Pricing tiers

	Starter	Growth (anchor)	Pro
Price / month	TZS 45,000	TZS 95,000	TZS 180,000
Annual (pay 10 months)	TZS 450,000	TZS 950,000	TZS 1,800,000
Who it fits	Owner + 1–4 cleaners, mostly sofa/carpet	5–15 staff, mixed residential + fumigation	15+ staff, commercial contracts, multiple branches
Staff accounts	5	15	Unlimited
Bookings / month	150	600	Unlimited (fair use)
WhatsApp bot + web booking page	Yes	Yes	Yes
Mobile money deposits & SMS-paste reconciliation	Yes	Yes	Yes
Before/after photo proof	Yes	Yes	Yes
Auto-dispatch suggestions + map board	—	Yes	Yes
Recurring reminders & re-engagement	100 reminders	1,000 reminders	5,000 reminders

	Starter	Growth (anchor)	Pro
Revenue & staff performance reports	Basic	Full	Full + export
Commercial contracts (recurring office cleaning, invoices)	—	—	Yes
Custom domain, multi-branch	—	—	Yes
Support	WhatsApp, business hours	WhatsApp, priority	Named account manager

Add-ons: marketing broadcast credits (sold at Meta cost + 30%), SMS bundles, extra branch TZS 40,000/month, done-for-you catalog photo shoot TZS 100,000 one-off.

No transaction fee. Because each tenant is merchant of record, the platform does not skim payments; this is also a selling point against owners' fear of "someone else holding my money." Revisit a small optional fee only if the platform later becomes a licensed aggregator.

Collecting the subscription: monthly USSD push to the owner's number on the renewal date via the platform's own Selcom/AzamPay account, 3 retries over 5 days, then a 7-day grace period in `PAST_DUE` (bot keeps working, dashboard shows a banner), then `SUSPENDED` (bot replies with the owner's phone number so no customer is lost).

5.2 Unit economics (rough, to validate)

Cost to serve a Growth tenant per month is roughly TZS 8,000–15,000: shared infrastructure, paid template messages for reminders, SMS fallback and image storage. That leaves an ~85% gross margin at TZS 95,000. The real cost is onboarding labour; keep it under 3 hours per tenant. Break-even on a ~TZS 3–4 M monthly team-plus-infra budget is roughly 40–50 paying tenants. Treat these figures as estimates and re-check Meta and gateway rate cards before publishing prices.

5.3 Onboarding plan for non-tech-savvy owners

Goal: **live and taking a real booking within 48 hours of saying yes**, with the owner doing less than 30 minutes of work.

1. **Sign-up over WhatsApp, not a form.** The sales link opens a chat with the platform's own bot: business name, city, services, number of cleaners. A human onboarding agent takes over from there.

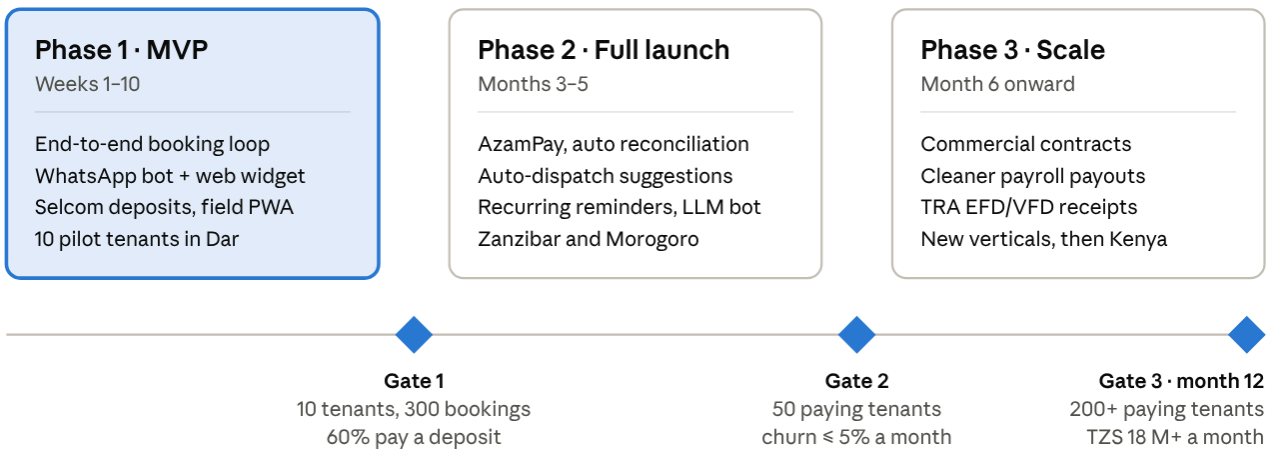
2. **We build the catalog for them.** Owners already post prices on Instagram and in WhatsApp status. The agent copies them (or the owner sends a voice note listing prices) and enters them. Seed a **template catalog per business type** (sofa/carpet, deep clean, fumigation, office) with typical Dar es Salaam prices so setup is "adjust, don't create."
3. **20-minute connect session** (in person in Dar, video call elsewhere): Embedded Signup for WhatsApp, gateway merchant credentials (or start the merchant application with them), add cleaners' phone numbers.
4. **Swap their links:** replace the wa.me link in the Instagram bio and highlights, pin a status post "Book in 1 minute", print a QR sticker for their van and flyers (we provide the design).
5. **Test booking together:** the agent books a fake sofa job on the owner's phone so they see what their customer sees, and the owner assigns it to a cleaner.
6. **Week 1 check-ins:** daily WhatsApp voice note from the agent for the first 5 days; fix any price or slot complaints the same day.
7. **Trial:** 30 days free or first 20 completed bookings, whichever comes first. No card or payment details needed to start.
8. **Learning in their format:** 60–90-second Swahili screen-recording videos sent on WhatsApp, one per task ("Jinsi ya kumpangia mfanyakazi kazi"). No manuals.
9. **Cleaners onboard themselves:** they receive an SMS link, enter OTP, and see a 3-screen picture tutorial. No training session required.

Acquisition levers: target businesses already visible on Instagram (they have demand and feel the DM pain), start with Dar es Salaam (Mbezi, Kinondoni, Kimara, Mikocheni clusters), then Zanzibar and Morogoro. Offer referral credit of one free month per referred paying business. Partner with cleaning-chemical and equipment distributors who already visit these owners. Run a monthly WhatsApp community for owners with business tips — it reduces churn and becomes a sales channel.

Pilot pricing: first 10 businesses get Growth at TZS 50,000/month for 6 months in exchange for weekly feedback and a testimonial video.

6. Development roadmap

Three phases over about 9 months: a 10-week MVP with 10 pilot tenants, a full launch by month 5, and scale from month 6. Each phase has an exit gate; do not start the next phase until it is met.



roadmap · 3 phases, 3 gates

Phase 1 is the current focus; the gate criteria under each diamond are the exit tests detailed below.

Phase 1 — MVP (weeks 1–10)

Scope is the end-to-end loop for **sofa/carpet/mattress and deep-cleaning businesses in Dar es Salaam**, Swahili + English.

Weeks	Milestone	Done when
1–2	Foundations	Monorepo, CI (GitHub Actions), Postgres + RLS + tenant interceptor with cross-tenant test, phone OTP auth, `tenants` / `users` / `memberships`, deploy pipeline to staging
2–3	Catalog + quote engine	All 6 `pricing_type`s, add-ons, distance-band travel fee; quote engine 100% unit-tested with real price lists from 3 target businesses
3–4	Availability + bookings	Slot generator (working hours × staff capacity × duration), 15-min Redis hold, booking lifecycle state machine with transition guards, per-tenant ticket numbers (UBC-2026-00042), booking source field, idempotent create_booking used by every channel

Weeks	Milestone	Done when
4–6	WhatsApp bot	Meta app + Embedded Signup, webhook → queue → state machine, list/button/location messages, WhatsApp Flow for items, 6 approved templates, human hand-off, dashboard inbox
5–6	Web booking widget	`/b/:slug` with live quote, map pin, slots, OTP; staff Create booking form; Track My Booking page with phone-code verification
6–7	Payments	`SelcomProvider` (push + checkout link + webhook + polling), `ManualProvider`, SMS-paste parser for M-Pesa/Mixx/Airtel/HaloPesa formats, cash records
7–8	Dispatch + field PWA	Today board (list + simple time grid), manual assign, field PWA with status buttons, camera-only photos, offline queue
8–9	Notifications + basic reports	Reminders, completion collage, rating, daily owner summary, revenue-by-day report
9–10	Super admin + billing + hardening	Tenant create/suspend, subscription push billing, Sentry, backups tested, load test 50 concurrent bot conversations

Out of MVP on purpose: auto-dispatch scoring, recurring contracts, custom domains, AzamPay, LLM intent parsing, native apps, multi-branch.

Exit gate: 10 tenants live, ≥ 300 bookings through the system, ≥ 60% of bookings with a mobile-money deposit, no cross-tenant data incident, ≥ 7 of 10 pilots willing to pay full price.

Phase 2 — Full launch (months 3–5)

- AzamPay provider + per-tenant gateway choice; automatic daily gateway-report reconciliation.
- Auto-dispatch suggestions (score formula in 3b), drag-and-drop time × staff grid, map board.
- Recurring maintenance engine (`recurrence_days`) + one-tap re-book; marketing broadcasts with opt-in tracking.
- LLM-assisted free-text understanding in the bot (Claude Haiku-class model) with a strict JSON schema output: `{services:[{name, qty, variant}], date_hint, area_hint}` ; confidence below threshold → normal menu.

- Custom domains via Cloudflare for SaaS; branding editor; Google review prompt.
- Fumigation/pest-control specifics: chemical used, re-entry time and safety notes on the job sheet and customer message.
- Self-serve sign-up path (still with optional assisted onboarding); public pricing page; referral credits.
- Expansion: Zanzibar (Unguja) and Morogoro.

Exit gate: 50 paying tenants, monthly logo churn ⩽ 5%, onboarding ⩽ 3 agent-hours per tenant, support tickets ⩽ 2 per tenant per month.

Phase 3 — Scale (month 6 onward)

- **Commercial module:** contracts for offices/hotels with recurring visit schedules, site checklists, monthly invoicing, supervisor sign-off on the cleaner's phone.
- **Payroll helper:** per-job/daily pay calculation for cleaners from completed jobs, with bulk mobile-money disbursement via the gateway's B2C/payout API.
- **Quality & insights:** photo-based quality review queue, cleaner leaderboard, customer lifetime value, churn prediction for recurring customers.
- **Native Android app** for field staff only if the PWA hits limits (background GPS, camera reliability).
- **TRA EFD/VFD receipt integration** for tenants that need tax receipts.
- **Infrastructure:** move to managed containers (e.g. Fly.io or AWS ECS), read replica for reports, per-tenant rate limits, SOC-style access logging; consider af-south-1 if latency matters.
- **Adjacent verticals on the same core:** laundry/dry-cleaning pickup, AC servicing, gardening, home repairs; then Kenya and Uganda (M-Pesa Daraja, MTN MoMo adapters).

Exit gate (12 months): 200+ paying tenants, ~TZS 18 M+ monthly recurring revenue, positive contribution margin after onboarding costs.

7. Build instructions for a developer or AI agent

Start with the repo skeleton and the first ten tasks below, in order; each is small enough for one pull request and one agent session.

7.1 Repository structure

```
safi360/
├─ apps/
│  ├─ api/                 # NestJS
│  │  └─ src/modules/{auth,tenants,catalog,quotes,availability,bookings,
│  │
dispatch,field,customers,payments,messaging,billing,reports,platform}/
│  │       each: *.controller.ts, *.service.ts, *.module.ts, dto/, *.spec.ts
│  ├─ worker/              # BullMQ processors: whatsapp-inbound, payments-
poll, notifications, daily-close
│  └─ web/                 # Next.js: /dashboard, /b/[slug], /field (PWA),
/platform
├─ packages/
│  ├─ db/                  # prisma/schema.prisma, migrations, rls.sql, seed
(template catalogs)
│  ├─ pricing/             # pure quote engine, zero deps, 100% tested
│  ├─ payments/            # PaymentProvider interface + Selcom, AzamPay,
Manual adapters + SMS parser
│  ├─ whatsapp/            # typed Cloud API client, message builders,
webhook verifier
│  ├─ i18n/                # sw.json, en.json — every user-facing string
│  └─ shared/              # zod schemas, enums, phone normaliser, money
helpers
├─ infra/                  # docker-compose.yml, Caddyfile, backup scripts
└─ docs/                   # ADRs, bot state diagram, template texts
```

7.2 Conventions (give these to any agent verbatim)

- TypeScript strict mode; zod schemas in `packages/shared` validate every API input and every webhook payload.
- Money = integer TZS, field names end in `_tzs`. Phone = E.164 string, normalised by `normalizeTzPhone()` at every boundary.
- Every tenant table has `tenant_id`; never pass `tenant_id` from the client — derive it server-side (JWT, slug, phone_number_id, payment reference).
- All webhook handlers: verify signature → persist raw payload → enqueue → return 200. Idempotency by provider message/transaction id.
- Booking status changes only through `BookingStateMachine.transition(booking, event, actor)`, which writes a `job_events` row and emits a domain event (`booking.confirmed`, `booking.completed`) that messaging and notifications subscribe to.
- Timezone: store UTC, render `Africa/Dar_es_Salaam` (UTC+3, no daylight saving).

- Every user-facing string goes through i18n with `sw` as the default; no hard-coded text in components or bot messages.
- Secrets (WhatsApp tokens, gateway keys) encrypted at rest; never logged. Redact phone numbers in logs to `+25575****123`.

7.3 First ten tasks

- [ ] Scaffold the monorepo (pnpm, Turborepo, NestJS api, Next.js web, worker), Docker Compose with Postgres 16 + PostGIS + Redis, GitHub Actions running lint + test.
- [ ] Prisma schema from Section 2.4, migration, `rls.sql` applying policies to all tenant tables, and an e2e test proving tenant A cannot read tenant B.
- [ ] `normalizeTzPhone`, money helpers and zod enums in `packages/shared` with tests (inputs: `0754…`, `754…`, `255754…`, `+255 754 …`).
- [ ] Phone OTP auth (WhatsApp template with SMS fallback via a stub provider), JWT with `tenantId` + `role`, `can(role, permission)` guard.
- [ ] `packages/pricing`: `quote(input, catalog, zones, now) → { lines, subtotal, travelFee, discount, total, durationMin }` covering all six pricing types; fixtures from real price lists.
- [ ] Catalog CRUD API + dashboard screens; seed template catalogs for sofa/carpet, deep cleaning, fumigation, office.
- [ ] Availability service: `getSlots(tenantId, serviceItems, date range, location) → Slot[]` respecting working hours, staff skills, existing assignments and travel buffer (default 45 min between jobs).
- [ ] Booking state machine + Redis slot hold + booking API used by both widget and bot.
- [ ] `packages/whatsapp`: webhook verify/route, message builders (text, buttons, list, location request, Flow, template), and the bot state machine for Section 4.1 with a fake transport for tests.
- [ ] `SelcomProvider` against the sandbox: push, checkout link, webhook verify, status query; `ManualProvider`; SMS-paste parser with sample confirmation messages from M-Pesa, Mixx by Yas, Airtel Money and HaloPesa (collect real samples from pilot owners; formats change).

7.4 Risks and mitigations

Risk	Impact	Mitigation
Meta business verification or Tech Provider approval is slow	Bot launch slips 2–6 weeks	Apply in week 1; build against the test number; web widget ships first

Risk	Impact	Mitigation
Owner's number not eligible for coexistence	Owner refuses to lose WhatsApp Business app	Offer a second "bookings" number; keep inbox in dashboard
Gateway merchant onboarding takes weeks per tenant	Deposits not live for new tenants	`ManualProvider` + SMS-paste reconciliation from day one
Customers still haggle on price	Bot bypassed	"Talk to a person" hand-off plus owner-set discount codes; track hand-off rate per tenant
Cleaners share one phone or skip photos	Proof-of-work breaks	Team-lead model (one phone per crew); completion blocked without photos
Owners churn after trial	Revenue	Daily owner summary and "bookings this month via Safi360" counter make the value visible; pilot price lock
Personal data rules	Regulatory	PDPC registration, Swahili privacy notice, data deletion endpoint per customer

Open questions to settle with pilot businesses: typical deposit customers will accept (20% vs 30%), whether crews are paid per job or monthly, and how many already hold a Selcom/AzamPay merchant account.